ARTCRAFT FLEET TECHNICAL BROCHURE · SERIES #16

AppCraft v0.1.0

AppCraft is the autonomous stigmergic application factory for the sandraschi fleet. It synthesizes local-first, zero-runtime-tax native desktop and web applications from structured domain blueprints (powered by DatabaseCraft).



v0.1.0	4 Crates	3890	6-Gate
RELEASE VERSION	WORKSPACE TIERS	PURE-RUST LOC	ARTCRAFT VERIFIED

Repository: `D:\Dev\repos\appcraft` · **Standard:** `CRAFT_BROCHURE_STANDARD.md` · **Compiled via:** `BookCraft v0.5.0`

AppCraft is the autonomous stigmergic application factory for the sandraschi fleet. It synthesizes local-first, zero-runtime-tax native desktop and web applications from structured domain blueprints (powered by **DatabaseCraft**).

Key Architecture

- **Stigmergic Coordination**: Decentralized environment-mediated task routing (`_stigmergy_ledger`).
- **Zero-Runtime-Tax**: Standalone single-binary `.exe` packaging with embedded SQLite B-Tree transactional storage.
- **Universal 3-Tier ArtCraft Standard**:
- `crates/appcraft-core`: Universal Command Bus (`Command::dispatch`).
- `crates/appcraft-templates`: Architecture starter stacks (Desktop egui, FastAPI React, Axum Webview).
- `crates/appcraft-mcp`: Hand-rolled line-delimited MCP stdio server + CLI (`appcraft-cli`).
- `crates/appcraft-app`: Native immediate-mode Studio IDE (`appcraft`).
- **Standard Control Port**: `14115`.

Status — does today / next week / non-goals

Does today: 4-crate workspace (templates as a crate), dispatch wired to DatabaseCraft blueprints; surgery blueprint → 6 tables + 12 ledger tasks, proven by test (3 fns); stigmergy ledger; 3 surfaces. Templates are manifests without bodies; CompileArtifact is a receipt stub (see hub assessment — honest, not finished).

Next week: template bodies v1 (`desktop_egui` CRUD skeleton + golden tree test); honest compile (real `cargo build` or rename); generate `bohr-ordination` + smoke + recipe.

Non-goals: factory generates files, doesn't host a platform; no second template before the first compiles.

Lineage — the Storytold craft-app pattern

Storytold ArtCraft (storytold/artcraft — "the IDE for artists") was the starting point. The 16 craft apps in the Storytold stable each ship one binary that is both the human CLI and the agent MCP server (stdio), headless-first, with `run.json` + report artifacts. That modus operandi and harness is what this repo implements; the code here is independent, the pattern is shared.

Product Requirements & Architecture Specification

Repo: `appcraft` (Pure-Rust ArtCraft 3-tier workspace)

Version: 0.1.0 (workspace.package)

Control port: 14115 (`appcraft --control 14115`)

Crates: `appcraft-core` (L1 bus) / `appcraft-templates` (starter manifests) / `appcraft-mcp` (L2 CLI + stdio) / `appcraft-app` (L3 egui Studio)

1. Problem

Fleet app ideas start as domain blueprints (DatabaseCraft tables) and die as one-off scripts. There is no factory that turns a blueprint into a runnable local-first desktop or web app with storage, ledger tasks, and a control surface wired on day one.

2. Solution

AppCraft is the autonomous stigmergic application factory. It consumes a structured domain blueprint (via `databasecraft-core` / `databasecraft-store`), emits a starter stack (desktop egui or web), and coordinates follow-up work through an environment-mediated `_stigmergy_ledger` instead of direct agent-to-agent calls.

3. Scope

- L1 `appcraft-core`: universal `Command::dispatch`, blueprint intake,

stigmergy ledger helpers, SQLite B-Tree storage via DatabaseCraft.

- `appcraft-templates`: architecture starter stacks (desktop egui, web).

Today these are manifests without bodies; bodies are the next milestone.

- L2 `appcraft-mcp`: hand-rolled line-delimited MCP stdio server + CLI

(`appcraft-cli`). Stdout is pure JSON-RPC; logs go to stderr.

- L3 `appcraft-app`: native immediate-mode Studio IDE (`appcraft` binary).

4. Does today / next week / non-goals

Does today: 4-crate workspace, dispatch wired to DatabaseCraft blueprints; surgery blueprint yields 6 tables + 12 ledger tasks (proven by tests); three surfaces (CLI, MCP, app shell) share one bus.

Next week: template bodies v1 (desktop egui CRUD skeleton + golden tree test); honest compile path (real `cargo build` or renamed receipt); first generated recipe with smoke check.

Non-goals: AppCraft generates files, it does not host a platform. No second template before the first one compiles. No cloud deploy, no auth service, no multi-tenant story.

5. Safety and honesty

- Zero-runtime-tax single binary; embedded SQLite, no sidecar ports for the desktop path (control port is opt-in for external tooling).
- `CompileArtifact` is currently a receipt stub, not proof of a build. It is labelled as such until the real `cargo build` path lands.
- No stubs presented as features in MCP `tools/list`.

6. Gates

`cargo clippy --workspace --all-targets --all-features -- -D warnings`, `cargo test --workspace`, `cargo fmt --all --check`, byte-exact MCP stdio probe (`initialize` -> `tools/list` -> `tools/call`), `Cargo.lock` committed, no `target/` / `*.bak` / `*.exe` in `git status`.